PRESENTACIÓN

PROYECTO BBDD

GRUPO D

Elena González

Gian Marco

Ariela Astorga

Jaime Rubio

Dani Lavía

TABLA DE CONTENIDOS

01. FLUJO DE TRABAJO

02. ESTRUCTURA BBDD

03. MODELO ENTIDAD - RELACIÓN

04. MODELO LÓGICO DE LA BBDD

05. CREACIÓN DE TABLAS E INGRESO DE INFORMACIÓN

06. INFORMACIÓN INTERESANTE USANDO QUERYS

FLUJO DE TRABAJO

1 - Abrir cuenta en **Rende**r para el despliegue de la base de datos (servidor **Cloud** donde se aloja la **BBDD**)

2 - Descargar **PGAdmin** (**interfaz visual** para trabajar con la **BBDD**) en **Docker** y conectarnos a la base de datos con la información de conexión de **Render** (**Host - Port - Database - Username - Password**).

3 - Estudiar los **CSV** y los datos de cada uno.

4 - Discutir en grupo cómo **organizar la información y correlación entre las variables** en excel.

5 - Asignar **Primary Keys** y **Foreign Key** a las variables.

6 - Hacer **diagramas** de **Entidad Relacion** en **draw.io.**

7 - Crear un repositorio en **GitHub** para compartir la información y los archivos del proyecto.

8 - Crear un **entorno virtual** con todas las librerías necesarias.

9 - Crear varios **JNotebook** en la carpeta **utils** para cargar la información en la **BBDD** en **PostgreSQL** (software de base de datos que está ejecutandose en **Render**):

- Crear las **tablas** usando **querys**.
- Crear **funciones y querys** para procesar la información de los **.csv** y traducirla a lenguaje **SQL**.
- Crear las **query INSERT** para alimentar las tablas con datos.

10 - Unificar los **JNotebook** en los que trabajó cada uno en uno solo llamado **INSERT,** que **crea y carga** paso a paso todas las **tablas y datos** necesarios.

11 - Subir las **tablas con sus datos** a la **BBDD**.

12 - Hacer diagrama de **Modelo Logico** de la **BBDD** desde **PGAdmin.**

13 - Crear **consultas** para presentar **datos relevantes** (fichero querys-explotacion.sql).

14 - Se ha **ocultado** la **información** de **conexión** con **Render/PostgreSQL** usando un archivo **.env**.

ESTRUCTURA BBDD

Discusión grupal de la estructura de la BBDD, campos y relaciones

TABLAS

Alumnos		
id_alumno	**pk**	**INT**
nombre		VARCHAR
email		VARCHAR
id_promocion	**FK**	INT

CAMPUS		
id_campus	**pk**	INT
nombre_campus		VARCHAR

VERTICAL		
id_vertical	**pk**	INT
nombre_vertical		VARCHAR

PROMOCIÓN		
id_promocion	**PK**	**INT**
nombre_mes		VARCHAR
fecha_comienzo		DATE
id_campus	**FK**	INT
id_vertical	**FK**	INT

PROYECTOS		
id_proyecto	**PK**	INT
nombre_proyecto		**VARCHAR**
id_vertical	**FK**	INT

id_alumno	nombre	email
1	pepe	epe@pe.com

id_campus	nombre_campus
1	Madrid
2	Valencia
3	Bilbao

id_vertical	nombre_vertical
1	Full Stack
2	DS

id_promocion	nombre	fecha_comienzo
1	Septien	#########
2	Febrerc	#########

id_proyecto	nombre	id_vertical
1	EDA	
2	FLOTA	
3	WEB	

Datos de Entrada

PROFESORES		
id_profesor	**pk**	INT
nombre		VARCHAR
rol		VARCHAR
modalidad		VARCHAR
id_promocion	**FK**	INT
id_modalidad	**FK**	INT

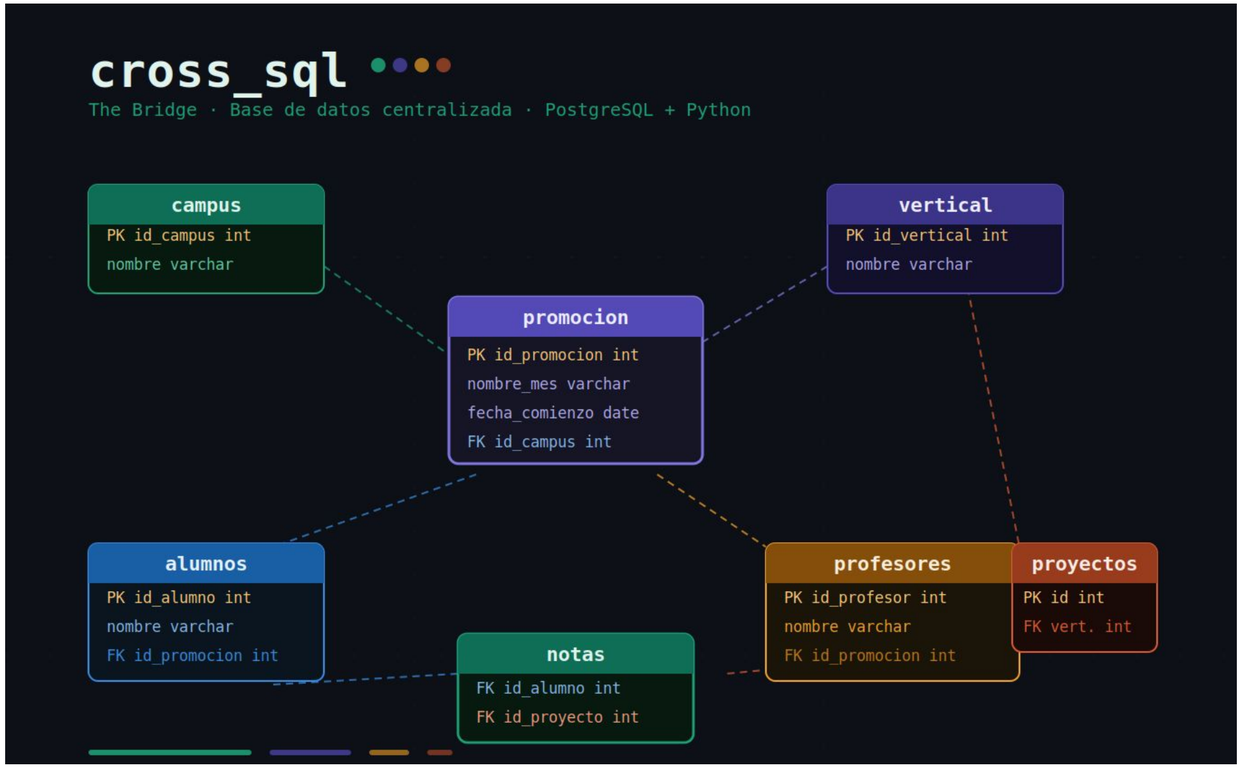
NOTAS		
id_alumno	**pk / fk**	INT
id_proyecto	**pk / fk**	INT
valor		VARCHAR
1	eda	apto
1	ml	no apto

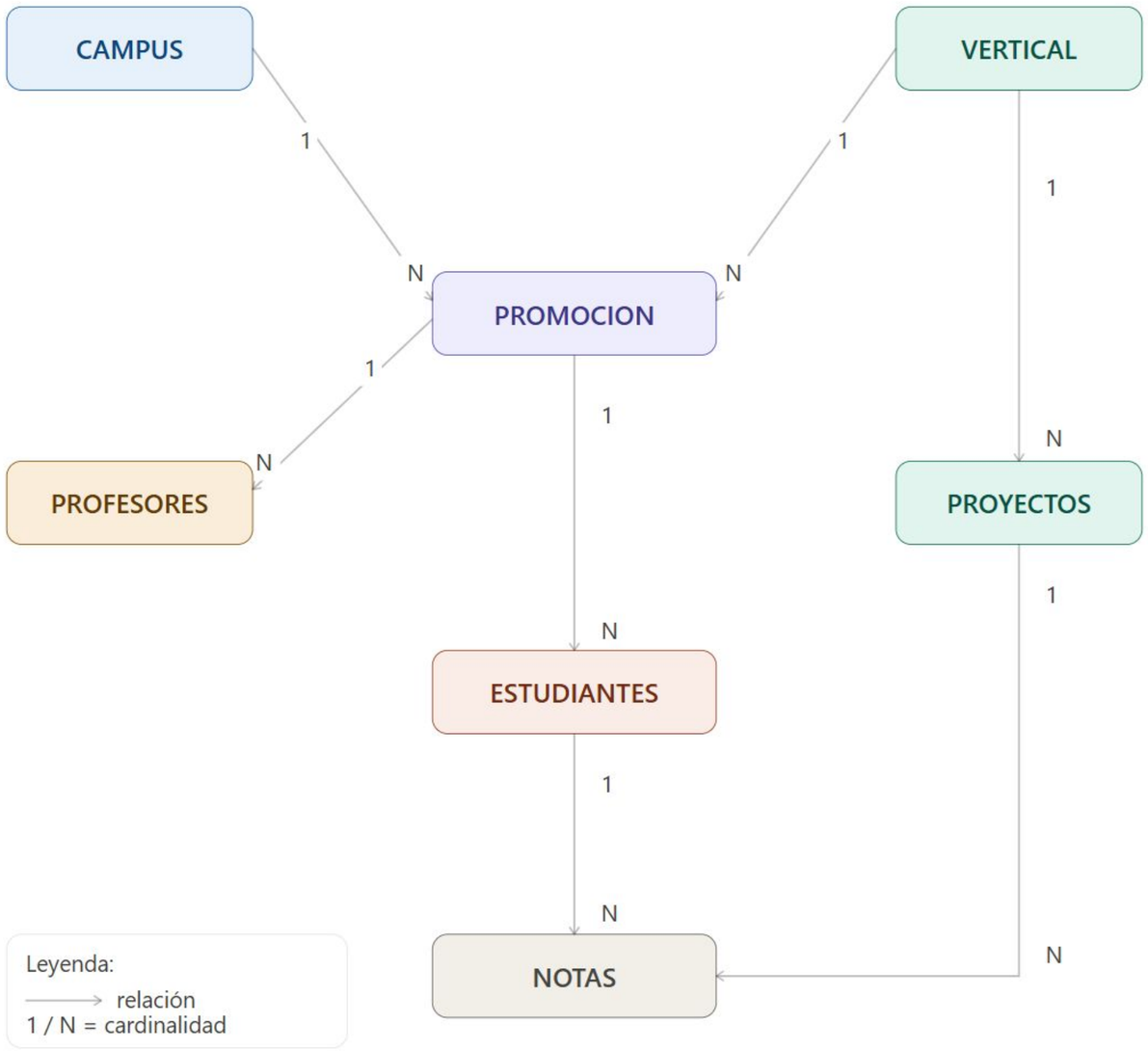
MODALIDAD		
id_modalidad	**pk / fk**	INT
valor	**pk / fk**	INT
1	eda	
1	ml	

Problemas encontrados:

- Cómo ingresar la variable modalidad, es necesaria una nueva tabla?
- id_campus nuevo en profesores (Valencia - febrero)
- Imposibilidad de conocer la promoción de un profesor

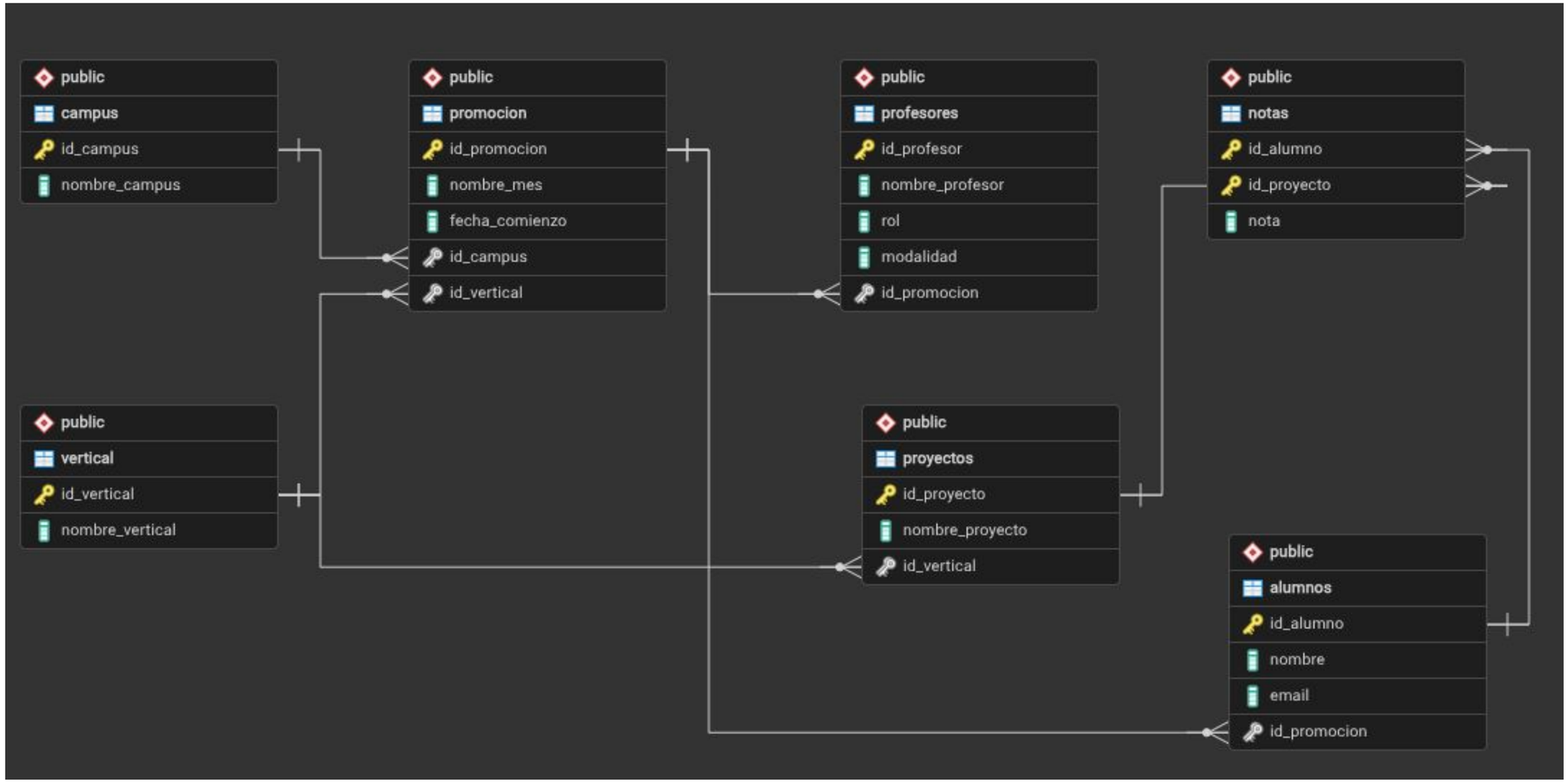
MODELO ENTIDAD - RELACIÓN





MODELO LÓGICO DE LA BBDD

Extraído desde PGAdmin



CONEXIÓN AL SERVIDOR Y BBDD

In [1]:

```
import pandas as pd
import os
from dotenv import load_dotenv
from typing import List, Optional
from sqlalchemy import String, Integer, Date, ForeignKey, PrimaryKeyConstraint, text, create_engine, inspect
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship
from sqlalchemy.orm import Session
from typing import Optional
from datetime import date
#cargar variables de conexion al servidor
load_dotenv()

usuario = os.getenv('DB_USER')
password = os.getenv('DB_PASSWORD')
host = os.getenv('DB_HOST')
puerto = os.getenv('DB_PORT')
nombre_db = os.getenv('DB_NAME')

engine = create_engine(f'postgresql://{usuario}:{password}@{host}/{nombre_db}')
# Prueba de conexión
try:
    with engine.connect() as connection:
        print("¡Conexión exitosa a PostgreSQL!")
except Exception as e:
    print(f"Error al conectar: {e}")
```

getenv lee las variables de entorno, en este caso las contraseñas del servidor de Render guardadas en el archivo **.env**

MOTIVACIÓN

CREACIÓN DE TABLAS Y CARGAR DATOS

```
#CAMPUS
query = '''
        CREATE TABLE campus (
        id_campus SERIAL PRIMARY KEY,
        nombre_campus VARCHAR(25) NOT NULL
        );
        '''
```

```
#Restricciones de no duplicidad en datos
query = '''
ALTER TABLE campus
ADD CONSTRAINT unique_campus_nombre
UNIQUE (nombre_campus);
'''
```

```
query = """
            INSERT INTO proyectos (nombre_proyecto, id_vertical) VALUES
            ('Proyecto_HLF', '1'),
            ('Proyecto_EDA', '1'),
            ('Proyecto_BBDD', '1'),
            ('Proyecto_ML', '1'),
            ('Proyecto_Deployment', '1'),
            ('Proyecto_WebDev', '2'),
            ('Proyecto_Frontend', '2'),
            ('Proyecto_Backend', '2'),
            ('Proyecto_React', '2'),
            ('Proyecto_FullStack', '2') ON CONFLICT (nombre_proyecto, id_vertical) DO NOTHING;
        """
```

INFORMACIÓN DE INTERÉS USANDO QUERYS

```
SELECT c.nombre_campus, COUNT(a.id_alumno) AS total_alumnos
FROM promocion p
JOIN alumnos a ON p.id_promocion = a.id_promocion
JOIN campus c ON p.id_campus = c.id_campus
JOIN vertical v ON p.id_vertical = v.id_vertical
GROUP BY c.nombre_campus
ORDER BY total_alumnos DESC
```

DEVUELVE LOS CAMPUS Y EL Nº TOTAL DE ALUMNOS

	nombre_campus character varying (25)	total_alumnos bigint
1	Madrid	39
2	Valencia	13

```
SELECT a.nombre, p.nombre_mes, p.fecha_comienzo, v.nombre_vertical
FROM alumnos a
JOIN notas n ON a.id_alumno = n.id_alumno
JOIN promocion p ON a.id_promocion = p.id_promocion
JOIN vertical v ON p.id_vertical = v.id_vertical
```

DEVUELVE LA TABLA CON EL NOMBRE DEL ALUMNO, SU FECHA DE INICIO Y LA VERTICAL A LA QUE PERTENECE.

	nombre character varying (150)	nombre_mes character varying (25)	fecha_comienzo date	nombre_vertical character varying (25)
1	Jafet Casals	Septiembre	2023-09-18	Data Science
2	Jorge Manzanares	Septiembre	2023-09-18	Data Science
3	Onofre Adadia	Septiembre	2023-09-18	Data Science
4	Merche Prada	Septiembre	2023-09-18	Data Science
5	Pilar Abella	Septiembre	2023-09-18	Data Science
6	Leoncio Tena	Septiembre	2023-09-18	Data Science
7	Odalys Torrijos	Septiembre	2023-09-18	Data Science
8	Eduardo Caparrós	Septiembre	2023-09-18	Data Science
9	Ignacio Goicoechea	Septiembre	2023-09-18	Data Science
10	Clementina Santos	Septiembre	2023-09-18	Data Science
11	Daniela Falcó	Septiembre	2023-09-18	Data Science
12	Abraham Vélez	Septiembre	2023-09-18	Data Science

INFORMACIÓN DE INTERÉS USANDO QUERYS

```
SELECT a.nombre, p.nombre_mes, p.fecha_comienzo, v.nombre_vertical,
    SUM(CASE WHEN n.nota = 'Apto' THEN 1 ELSE 0 END) AS total_aptos
FROM alumnos a
JOIN notas n ON a.id_alumno = n.id_alumno
JOIN promocion p ON a.id_promocion = p.id_promocion
JOIN vertical v ON p.id_vertical = v.id_vertical
GROUP BY a.nombre, p.nombre_mes, p.fecha_comienzo, v.nombre_vertical;
```

DEVUEVE LA CANTIDAD PROYECTOS APROBADOS POR ALUMNO, Y MUESTRA LA PROMOCIÓN EN LA QUE CURSA.

	nombre character varying (150)	nombre_mes character varying (25)	fecha_comienzo date	nombre_vertical character varying (25)	total_aptos bigint
1	Ernesto Uriarte	Febrero	2024-02-12	Full Stack	5
2	Maximiliano Menéndez	Septiembre	2023-09-18	Data Science	4
3	Isabel Ibáñez	Febrero	2024-02-12	Data Science	3
4	Amor Larrañaga	Septiembre	2023-09-18	Full Stack	4

MUCHAS GRACIAS POR SU ATENCIÓN

¿Alguna duda?

Siguenos en Github

Elena Gonzalez ---------- Elegm92

Ariela Astorga ---------- arielaastorga

Daniel Lavia ---------- danilaviaart-lgtm

Gian Marco Assandria ------ gianass-first

Jaime Rubio ---------- BV-Works